PROJECT DOCUMENTATION

Digi Store Support Platform — Project Documentation

Generated from DOCUMENTATION.md · 03 October 2026

A record of everything built so far: what the system does, how it is put together, the order in which it was built, the problems hit along the way, and what is still open.

For the choice of each technology and the alternatives considered, see TECH_STACK.md; for setup and API examples, see README.md.

1. Overview

The platform is an online shop back-end with an AI support assistant, for a fictional store, **Digi Store**. It sells products across several categories (electronics, clothing, books, home & kitchen, gift cards, and the **Acme Cloud Storage** subscription plans). It has three parts:

- **A store back-end** — REST resources for **product types, products (with variants), orders (with line items) and tickets**, stored in PostgreSQL, with customer/admin access control.
- **An agentic support assistant** — for each message, the language model decides what it needs and requests it through **tool calling**: searching the live catalog, looking up the signed-in customer's own orders and tickets, and — in two confirmed steps — **placing and cancelling orders**. Policy and how-to answers come from an FAQ, retrieved by semantic search.
- **Chat channels** — a stateless REST endpoint, a stateful **WebSocket**, and a **web chat page** that shows an interactive **order form** (pick a variant, set a quantity, see the total, confirm).

The model never touches the database and never writes SQL. Every tool runs a fixed, parameterised query scoped to the *authenticated* customer, never to an id the model supplies. The only actions that change data — ordering and cancelling — require a confirmation that the **server** checks, not just the model.

Layer	Technology
API	FastAPI + Uvicorn; class-based views for store resources; WebSocket chat
Web UI	Single-file HTML/JavaScript chat page served at `/`

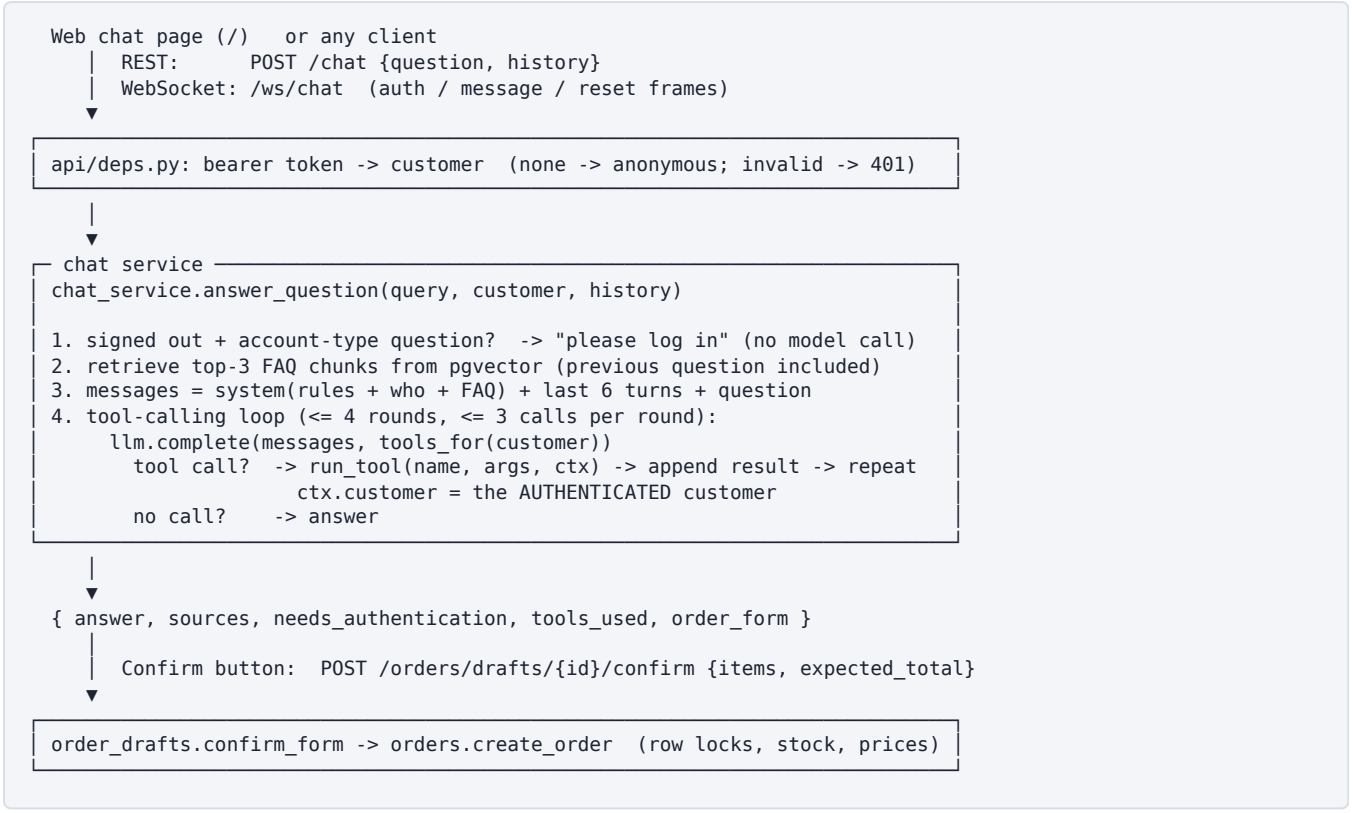
Layer	Technology
ORM / migrations	SQLAlchemy 2.0 models (psycopg2 driver), Alembic migrations 0001–0006
Authentication	Argon2id password hashing, JWT (HS256) bearer tokens, OAuth2 password flow, admin role
LLM	`Qwen/Qwen2.5-72B-Instruct` (pinned to `deepinfra`) via Hugging Face Inference Providers — **billed**, with multi-token failover
Agent behaviour	Tool calling (`app/services/tools.py`): 9 tools, 1–4 model round-trips per message
Embeddings	`BAAI/bge-small-en-v1.5` via `fastembed` (local, ONNX, 384 dims)
Database	PostgreSQL 16 with `pgvector`; JSONB for per-type attributes, variants and order drafts
Testing	pytest: scripted fake LLM + tests against a real, rolled-back Postgres (203 tests)
Packaging	Docker + Docker Compose

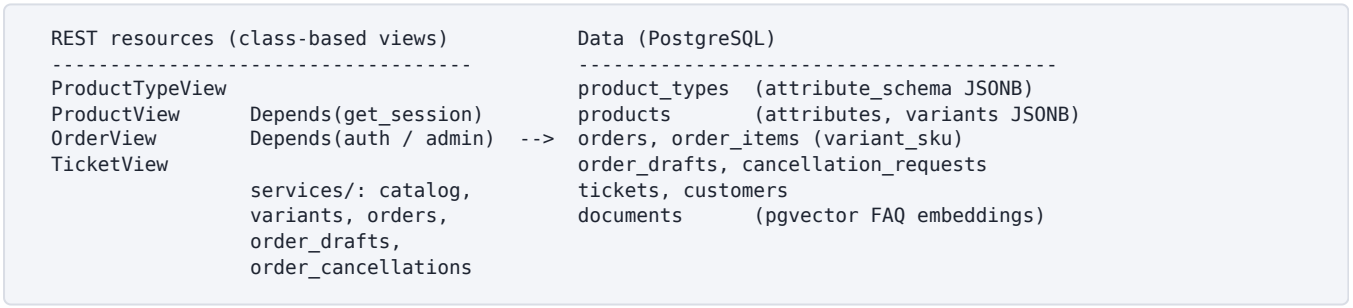
2. Build History (chronological)

#	Task	Outcome
1	**Project structure cleanup**	`chat.py` was misplaced in a stray `chunks/` folder; moved to the root. Created a `.venv`.
2	**Switch from Anthropic to an open model**	Replaced the Anthropic SDK with `huggingface_hub`'s `InferenceClient`. Auth moved to `HF_TOKEN`. (Later corrected: see #12 — no provider on the router is actually free.)
3	**Secrets handling**	`python-dotenv`, `.env` / `.env.example`, `.gitignore`.
4	**Fix HF 403**	Token lacked the *"Make calls to Inference Providers"* scope.
5	**Fix HF 400 (`model_not_supported`)**	Switched from Qwen2.5-7B to `microsoft/phi-4` after checking the live router model list.
6	**Database integration**	Postgres lookups of customers/orders/tickets and identifier extraction.
7	**Dockerization**	`Dockerfile` + `docker-compose.yml`; resolved host-port conflicts.
8	**FastAPI restructure + API endpoint**	Replaced the terminal loop with `POST /chat`; `app/api`, `app/schemas`, `app/services` layout.
9	**pgvector semantic retrieval**	Replaced TF-IDF with embedding search in Postgres.
10	**Authentication**	Login/registration, JWT tokens, argon2, per-customer data scoping.
11	**Models, class-based views, DB-backed answers**	SQLAlchemy models + Alembic; products / orders / tickets as class-based views with role-based access. Replaced `schema.sql` and the raw-SQL data layer. **Breaking schema change** — see §6.
12	**Tool calling**	Replaced "always fetch everything" with a model-driven tool-calling loop. Required switching models (`phi-4` cannot call tools — §11) to `Qwen2.5-72B-Instruct`. Corrected the project's own docs: the LLM is billed, not free. First automated tests (fake LLM).
13	**WebSocket chat + web interface**	`WS /ws/chat` (one connection = one conversation, server-side history, token sent in a frame and re-checked per message); a browser chat page at `/` using only the public API, switchable between WebSocket and REST.

#	Task	Outcome
14	**Conversation history**	`answer_question` accepts earlier turns (last 6); `POST /chat` takes an optional `history`. FAQ context moved into the system message so follow-ups ("how much is it?") resolve correctly.
15	**SQLAlchemy 2.1 driver fix**	A rebuild pulled SQLAlchemy 2.1, whose default `postgresql://` driver is psycopg 3 (not installed); the app stopped starting. `sqlalchemy_url()` now names `psycopg2` explicitly.
16	**E-commerce: product types**	`product_types` table with per-type attribute definitions (JSONB); products carry validated `attributes`; `/product-types` endpoints; seed catalog of 6 types and 30 products. Product search ranks by number of matching terms.
17	**FAQ rewritten for e-commerce**	20 sections: ordering, statuses, cancellation, stock, payment, shipping, digital delivery, returns, refunds, sizing, warranty, gift cards, accounts, support, cloud plans. No prices or stock in the FAQ — those come only from the live catalog.
18	**Rename to Digi Store**	Store name and support email (`support@digistore.example`) across the prompt, FAQ, UI and docs; product brands unchanged.
19	**Ordering by chat**	`prepare_order` / `confirm_order` tools; quotes stored in `order_drafts`; server refuses to confirm unless quoted in an earlier message, the latest message is a clear yes, it's < 15 min old and the total is unchanged.
20	**Cancelling by chat**	`request_cancellation` / `confirm_cancellation`, same two-step protection; `cancellation_requests` table; only the customer's own pending orders, even for admins.
21	**HF token failover + error logging**	`HF_FALLBACK_TOKENS`: on 401/402/403/429 the request moves to the next token and the failed one is skipped for a while. Provider errors are logged (customers still see a generic 503).
22	**Order form**	Chat answers carry an `order_form` (variants, prices, stock caps); the web page renders it with a live total; `POST /orders/drafts/{id}/confirm` places the order only if the total shown is still correct.
23	**Product variants (JSONB)**	`products.variants` (SKU, options, optional price, stock, active flag per variant); variant-flagged attributes on types; variant-aware ordering, stock, cancelling, search and order form; seed converted per-size products into variants.
24	**Technology documentation**	`TECH_STACK.md` rewritten: role, significance and reasons for each technology over its alternatives.

3. Architecture





Design principles

- **Retrieval is separate from generation.** FAQ text is fetched up front; the model sees only what it was given or explicitly requested.
- **The model decides what it needs; the server decides what it's allowed to have.** The model chooses *which* tool and *what* lookup arguments. It never chooses *whose* data: account tools read the customer from the verified token via `ToolContext`, never from the model's arguments.
- **The model never touches the database and never writes SQL.** Only the fixed tools in `tools.py` exist for it to call.
- **Authorization is enforced by what a tool can reach, not by what the model is told.** Signed-out callers aren't offered account tools; another customer's order looks identical to a nonexistent one.
- **Write actions need a confirmation the server verifies.** Ordering and cancelling are two-step; the checks (earlier turn, clear yes, expiry, unchanged total) are in code and covered by tests.
- **The server owns the money.** Clients send product/variant ids and quantities only; prices, totals, status and ownership are set server-side. The order form must echo the total it showed, or nothing is ordered.

4. Project Structure

```
support-bot/
├── app/
│   ├── main.py                    # FastAPI app; lifespan: migrate -> index FAQ -> build clients
│   ├── config.py                  # env-driven settings (fail-fast); HF token list; LLM model/provider
│   ├── db/  base.py  session.py (pooled engine, sqlalchemy_url)  migrate.py
│   ├── models/                    # customer, product (Product, ProductType), order (Order, OrderItem,
│   │                              #   OrderDraft, CancellationRequest), ticket, enums
│   ├── api/
│   │   ├── deps.py                # token -> customer; customer_from_token; require_admin; get_session
│   │   ├── routes/                # function-based: auth, chat (REST), chat_ws (WebSocket), health, web (/)
│   │   └── views/                 # class-based: base, product_types, products, orders, tickets
│   ├── schemas/                   # pydantic: auth, chat (incl. OrderForm, WS frames), product
│   │                              #   (incl. variants), product_type, order (incl. order form), ticket
│   ├── services/
│   │   ├── auth.py                # argon2 + JWT
│   │   ├── catalog.py             # attribute + variant validation against the product type
│   │   ├── variants.py            # variant lookup, price, stock, labels, SKU uniqueness, in-stock SQL
│   │   ├── orders.py              # create_order (variant-aware, locks), status transitions
│   │   ├── order_drafts.py        # bot quotes, "clear yes" check, order form confirm/discard
│   │   ├── order_cancellations.py # two-step cancellation
│   │   ├── database.py            # dict-returning helpers for tools and login
│   │   ├── llm.py                 # HF client: retry, multi-token failover, 503 on failure
│   │   ├── tools.py               # the 9 tools + their security model
│   │   ├── chat_service.py        # FAQ -> tool-calling loop -> answer (+ order form)
│   │   ├── customer_lookup.py     # signed-out gate (spots account-type questions)
│   │   └── vector_knowledge_base.py  embeddings.py  document_loader.py  errors.py
│   └── web/index.html             # browser chat page + order form
├── migrations/versions/           # 0001 ... 0006
├── knowledge/faq.md               # 20 FAQ sections
├── seed_data.py                   # demo accounts, 6 product types, catalog with variants, orders, tickets
├── tests/                         # conftest (rolled-back Postgres session) + 6 test files
├── Dockerfile  docker-compose.yml  requirements.txt  requirements-dev.txt  .env.example
└── README.md  DOCUMENTATION.md  TECH_STACK.md
```

5. Components in Detail

5.1 `app/main.py`

At startup: runs migrations, creates the pooled engine and session factory, indexes `knowledge/` into pgvector, and builds the `LLMClient` with every configured token. Registers the function-based routers (auth, chat, chat_ws, web, health), the class-based views (product types, products, orders, tickets), and maps `DomainError` subclasses to JSON errors with the right status (`Retry-After` on 503).

5.2 `app/config.py`

`get_settings()` fails fast if `HF_TOKEN`, `DATABASE_URL` or `JWT_SECRET` is missing or `JWT_SECRET` is under 32 characters. `hf_tokens` is `HF_TOKEN` followed by `HF_FALLBACK_TOKENS` (comma-separated, blanks and duplicates removed). Also `llm_model` / `llm_provider` (default `Qwen/Qwen2.5-72B-Instruct` on `deepinfra`).

5.3 Models (`app/models/`)

Model	Notes
`Customer`	`email` unique, `plan`, `password_hash` (nullable), `is_admin`, `created_at`
`ProductType`	`slug` unique, `name`, `description`, **`attribute_schema` JSONB** (attribute definitions, incl. `"variant": true` flags), `is_active`

Model	Notes
`Product`	`sku` unique, `product_type_id` FK (joined, inner-join load), **`attributes` JSONB**, `price ≥ 0`, `billing_period`, `stock_quantity` (NULL = unlimited; NULL when variants exist), **`variants` JSONB** (GIN index), `is_active`
`Order`	`customer_id` FK; `status ∈ {pending, processing, completed, cancelled}`; `total_amount`; `items`
`OrderItem`	`order_id` FK (cascade), `product_id` FK, **`variant_sku`**, `product_name` and `unit_price` **snapshotted** at purchase, `quantity > 0`
`OrderDraft`	A bot quote: `lines` JSONB, `total_amount`, `status ∈ {open, placed, superseded, expired}`, `order_id`, `created_at`/`expires_at` (app clock)
`CancellationRequest`	`order_id`, `status ∈ {open, done, superseded, expired}`, `created_at`/`expires_at`
`Ticket`	`customer_id` FK; `subject`, `description`; `status ∈ {open, in_progress, resolved, closed}`

Snapshotting name (including the variant, e.g. "Acme logo T-shirt (Size: M, Colour: Black)") and price means later changes never rewrite order history.

5.4 Migrations (`migrations/`)

Migration	Change
`0001_baseline`	Pre-Alembic tables (`customers`, `orders`, `tickets`), all `IF NOT EXISTS`
`0002_products_orders_tickets`	Products, order line items, admin flag, CHECK constraints; converts legacy orders into line items (irreversible)
`0003_product_types`	`product_types`; `products.product_type_id` and `attributes`; existing products moved to a `general` type
`0004_order_drafts`	Bot quotes
`0005_cancellation_requests`	Bot cancellation requests
`0006_product_variants`	`products.variants` JSONB + GIN index; `order_items.variant_sku`

Applied at startup (convenient for one instance; run `alembic upgrade head` as a deploy step with several). The pgvector `documents` table is excluded from autogenerate.

5.5 Class-based views (`app/api/views/`)

FastAPI has no class-based views, so `base.py` provides a small `APIView` + `@route`. Inbound schemas use `extra="forbid"` (unknown fields → 422).

View	Behaviour
`ProductTypeView`	Public reads of active types; admin create/PATCH/DELETE. Changing `attribute_schema` is refused (409) if existing products would no longer be valid; a type with products can't be deleted (409).
`ProductView`	Public reads of active products, filters `q`, `type`, `min_price`, `max_price`, `in_stock` (variant-aware); admin writes validate attributes and variants against the type, enforce SKU uniqueness across products **and** variants (advisory locks), and require product stock to be NULL when variants exist. Validation share-locks the type row.
`OrderView`	Customers create/see their own orders (lines take `variant_sku`); admins see all; cancel; admin status changes; **order form** `POST /orders/drafts/{id}/confirm` and `/discard`.

View	Behaviour
`TicketView`	Customers open/see/close their own tickets; admins see all and set status.

5.6 Tool calling (`app/services/tools.py`)

`run_tool` executes one model-requested call and never raises — every failure becomes a JSON error the model reads.

Tool	Login	What it does
`search_products`	No	Keyword (name, description, SKU, attribute values, variant options, type), category, max price, in-stock search, ranked by matching terms; lists variants with their SKU, options, price and availability; names sold-out matches when filtering to in-stock
`list_product_types`	No	Active categories with product counts and the details each lists
`get_order`	Yes	One of the caller's orders by number
`list_my_orders`	Yes	The caller's orders, optional status filter
`list_my_tickets`	Yes	The caller's tickets, optional status filter
`prepare_order`	Yes	Quote items (product or variant SKUs + quantities); saves a draft; returns a summary to the model and an **order form** to the client
`confirm_order`	Yes	Place the caller's open quote — refused unless the latest message is a clear yes (plus the draft checks, §5.12)
`request_cancellation`	Yes	Check one of the caller's orders can be cancelled; record the request
`confirm_cancellation`	Yes	Cancel it — refused unless the latest message is a clear yes

Security model: no argument schema contains a customer identifier (a test asserts no tool parameter even mentions "customer"); `ToolContext.customer` comes from the verified JWT; `tools_for(customer)` filters what's offered and `run_tool` re-checks; another customer's order returns the same "not found" as a nonexistent one; tool results are labelled as data, not instructions.

Robustness: pydantic argument validation; `_inline_refs` + `_clean_schema` produce small, self-contained schemas (no `$ref`, `title`, `anyOf`); search-term cleaning; result size cap; the order form is popped out of the tool result so it reaches the client, not the model's context.

5.7 The chat loop (`chat_service.answer_question`)

1. **Login gate** — signed out + account-type question (regex) → login prompt, no model call.
2. **FAQ retrieval** — top 3 chunks; for follow-ups the previous question is searched too.
3. **Messages** — system message (rules, who is signed in, FAQ text), then up to 6 earlier turns, then the question alone. (Keeping the FAQ out of the latest user message was needed for follow-ups to resolve against the conversation.)
4. **Loop** — ≤ 4 rounds, ≤ 3 calls per round, exact repeats short-circuited; at the cap, one final tools-off call.
5. **Result** — `answer`, `sources`, `needs_authentication`, `tools_used`, `order_form`. If a form is attached and the model's reply is near-empty, a standard sentence pointing at the form is used.

The system prompt includes a worked example so the model shows the order form straight away instead of first asking "which size?".

5.8 `app/services/llm.py`

Several tokens, tried in order. A 401/402/403/429 on a token moves the same request to the next token and benches the failed one (1 hour; 1 minute for 429). If all are benched, all are tried anyway (soonest-available first). Transient 5xx/network errors are retried once on the same token; other failures become 503. Every provider error is logged with the token's last 4 characters only. `temperature=0.2`.

5.9 Authentication

Argon2id hashing (timing-equal for unknown users); JWT HS256 with required `exp`/`sub`; invalid tokens → 401 (never silently anonymous); `require_admin` → 403; registration can't set `is_admin` or claim an existing email. The WebSocket uses the same token check (`customer_from_token`).

5.10 `app/services/orders.py`

`create_order(session, customer_id, [(product_id, variant_sku, quantity)])` merges duplicate lines, caps 100 per line, locks product rows (`FOR UPDATE OF products`, id order), requires a variant for products that have them, refuses unknown/inactive products or variants (including a variant SKU the product doesn't have) with one message, checks and changes stock on the product or variant, snapshots name and price, and never commits. Cancelling returns stock to the right variant. Status transitions: admin `pending → processing | cancelled`, `processing → completed | cancelled`; customers may cancel their own pending orders.

5.11 Catalog and variants (`catalog.py`, `variants.py`)

- **`catalog.validate_product`** checks a product against its type: without variants, all attributes on the product; with variants, variant-flagged attributes in each variant's `options` and the rest on the product; required/unknown/kind/choice checks; unique variant SKUs and option combinations; max 50 variants. Returns the stored JSON form (SKUs upper-case, prices as strings).
- **`variants.py`** — `resolve_sku` (product or variant SKU → product + variant, via a GIN-indexed containment query), `unit_price`, `stock_of`, `change_stock` (replaces the JSON list so the change is saved), `display_name`, `sellable_items`, `in_stock_clause` (SQL using `jsonb_path_exists`), `lock_skus` (advisory locks) and `sku_taken`.

5.12 Ordering by chat (`order_drafts.py`)

- **`prepare_draft`** resolves SKUs (a base product's SKU preselects an in-stock variant), checks stock, supersedes the customer's previous open draft, saves the quote, and builds the **order form** (each line's variants with name, price, billing period and max quantity).
- **`confirm_draft`** (typed "yes"): latest open draft, locked; refused if expired, created in the same request, or if the total changed (a savepoint undoes the stock taken).
- **`confirm_form`** (button click): the customer's chosen SKUs and quantities; must be their own open, unexpired draft; refused (409, nothing saved) if the total differs from `expected_total`. A click is the confirmation, so no earlier-turn rule.
- **`is_confirmation(message, action)`** — strict English pattern; "no", "wait", "change", "keep"… mean no; "cancel" means no when confirming an order but yes when confirming a cancellation.

5.13 Cancelling by chat (`order_cancellations.py`)

`request_cancellation` (own order, still pending; explains processing/completed/cancelled cases) and `confirm_cancellation` (latest open request, locked; earlier turn, not expired; `transition_order` — refused if the order moved on meanwhile). Orders are always looked up by the authenticated customer's id, so even admins can only cancel their own orders through the bot.

5.14 WebSocket chat (`routes/chat_ws.py`)

Per-connection `Conversation`: token, customer, history (≤ 6 turns). Frames are validated with a discriminated pydantic union; oversized, binary or invalid frames get an `invalid_frame` error and the socket stays open. The token is re-checked before each question; an expired token produces `unauthorized` and a sign-out, never an anonymous answer. Changing identity clears the history. Blocking work runs in the thread pool.

5.15 Web chat page (`app/web/index.html`)

Sign in / register, chat over WebSocket (auto-reconnect with backoff) or REST (sends recent history), source and tool chips, and the **order form** card: variant picker with prices (sold-out options disabled), quantity capped at stock, live line and order totals (computed in cents), Confirm / Cancel. A newer form retires older ones. Token kept in `sessionStorage`; light/dark aware; phone-width layout.

5.16 Retrieval stack

`document_loader.py` chunks `.md` by `##`; `embeddings.py` runs `bge-small-en-v1.5` locally; `vector_knowledge_base.py` self-provisions the `vector` extension, `documents` table and HNSW cosine index, re-indexes `knowledge/` at startup, and returns top-k by cosine similarity (own psycopg2 connections).

6. Data Model

Table	Columns
`customers`	`id`, `name`, `email` (unique), `plan`, `password_hash`, `is_admin`, `created_at`
`product_types`	`id`, `slug` (unique), `name`, `description`, `attribute_schema` JSONB, `is_active`, timestamps
`products`	`id`, `sku` (unique), `product_type_id → product_types`, `name`, `description`, `attributes` JSONB, `price`, `billing_period`, `stock_quantity`, `variants` JSONB (GIN), `is_active`, timestamps
`orders`	`id`, `customer_id → customers`, `status`, `total_amount`, timestamps
`order_items`	`id`, `order_id → orders (cascade)`, `product_id → products`, `variant_sku`, `product_name`, `unit_price`, `quantity`
`order_drafts`	`id`, `customer_id`, `lines` JSONB, `total_amount`, `status`, `order_id`, `created_at`, `expires_at`
`cancellation_requests`	`id`, `customer_id`, `order_id`, `status`, `created_at`, `expires_at`
`tickets`	`id`, `customer_id → customers`, `subject`, `description`, `status`, timestamps
`documents` (app-managed)	`id`, `source`, `content`, `embedding VECTOR(384)` — rebuilt from `knowledge/` at startup

A variant (an element of `products.variants`):

```
{"sku": "TSHIRT-LOGO-BLK-M", "options": {"size": "M", "color": "Black"},
 "price": null, "stock_quantity": 30, "is_active": true}
```

Demo data (`seed_data.py`, idempotent):

Accounts	`admin@example.com` (admin), `ada@example.com`, `grace@example.com` — random passwords, printed once; never promotes a pre-existing account
Product types	Digital services, Electronics, Clothing, Books, Home & kitchen, Gift cards (+ the migration's `general`)
Products	25 active; 5 with variants (14 variants): Acme logo T-shirt (black/white × M/L), Zip hoodie (3), Acme Backup Drive (1TB/4TB), Digi Store gift card ($25/$50/$100), The Backup Handbook (paperback/e-book). One deliberately sold-out product (USB-C hub) and one inactive (kettle).
Conversion	On an existing database, earlier per-size products became variants (stock carried over); the old rows stay inactive as `…-OLD<id>` for order history. Types created before variants are upgraded if their products stay valid.

7. API Reference

7.1 Authentication

- `POST /auth/register` — `{name, email, password}` → `201` profile; `409` if the email exists.
- `POST /auth/login` — form `username` (email) + `password` → `{access_token, token_type, expires_in}`; one identical `401` for every failure.
- `GET /auth/me` — the caller's profile.

7.2 Store resources

Endpoint	Signed out	Customer	Admin
`GET /product-types`, `GET /product-types/{id}`	active	active	+ inactive
`POST/PATCH/DELETE /product-types…`	401	403	yes
`GET /products`, `GET /products/{id}`	active	active	+ inactive
`POST/PATCH/DELETE /products…`	401	403	yes
`POST /orders`	401	for yourself	yes
`GET /orders`, `GET /orders/{id}`	401	own	all
`POST /orders/{id}/cancel`	401	own, while pending	any cancellable
`PATCH /orders/{id}/status`	401	403	yes
`POST /orders/drafts/{id}/confirm`, `/discard`	401	own draft	own draft
`/tickets…`	401	own	all

Placing an order with a variant:

```
POST /orders
{ "items": [ {"product_id": 12, "variant_sku": "TSHIRT-LOGO-BLK-M", "quantity": 2} ] }
```

Confirming a chat order form:

```
POST /orders/drafts/82/confirm
{ "items": [ {"sku": "TSHIRT-LOGO-BLK-L", "quantity": 2} ], "expected_total": "48.00" }
```

Errors: `422` (validation, unknown/inactive product or variant, missing variant), `409` (stock, status transition, duplicate SKU, price changed, used/expired form).

7.3 Chat

`POST /chat` — `{question, history?}` (history: up to 20 `{question, answer}`, last 6 used). Response:

```
{ "answer": "...", "sources": ["faq.md"], "needs_authentication": false,
  "tools_used": ["search_products", "prepare_order"], "order_form": { ... } | null }
```

`WS /ws/chat` — client frames: `auth {token}`, `message {question}`, `reset`, `ping`. Server frames: `ready`, `auth_ok`, `typing`, `answer` (same fields as `POST /chat`), `reset_ok`, `pong`, `error {code, detail}` with codes `unauthorized`, `invalid_frame`, `service_unavailable`, `internal_error`.

Caller	Message	Result
Signed out	general / FAQ	Answered from the FAQ
Signed out	product question	`search_products` / `list_product_types` only
Signed out	mentions "my order", an order number or an email	`needs_authentication: true`, no model call
Signed in	anything	FAQ + any tool, scoped to that customer
Signed in	"I'd like to order a hoodie"	Quote + order form; placed only after a clear yes or the Confirm button
Any	provider failing	`503` + `Retry-After` (cause in the server log)

7.4 Other

`GET /` chat page · `GET /health` · `/docs` Swagger UI · `/openapi.json`.

8. Configuration

Variable	Purpose
`HF_TOKEN`	Hugging Face token with "Make calls to Inference Providers". **Billed.**
`HF_FALLBACK_TOKENS`	Optional, comma-separated extra tokens (ideally other accounts) used on 401/402/403/429
`LLM_MODEL` / `LLM_PROVIDER`	Default `Qwen/Qwen2.5-72B-Instruct` / `deepinfra`; the pair must support tool calling
`JWT_SECRET`	Required, ≥ 32 characters; changing it logs everyone out

Variable	Purpose
`JWT_EXPIRE_MINUTES`	Token lifetime, default 60
`DATABASE_URL`	Set by Compose for Docker; plain `postgresql://` (the app adds `+psycopg2` for SQLAlchemy)

`.env` is git- and docker-ignored. After editing `.env`, run `docker compose up -d --force-recreate app` (a plain restart doesn't re-read it).

9. Running the Project

```
cp .env.example .env                            # set HF_TOKEN and JWT_SECRET
docker compose up -d                            # db + app; migrations run at startup
docker compose run --rm app python seed_data.py   # demo data; copy the printed passwords
```

Chat page and API at `http://localhost:8020`. Code edits reload automatically; **FAQ edits** need `docker compose restart app` to be re-indexed.

Tests (fake LLM; database tests use the running Postgres inside rolled-back transactions and skip without it):

```
docker compose run --rm app sh -c "pip install -q -r requirements-dev.txt && python -m pytest -q"
```

Cost note: every chat message spends HF credits (more when tools are used). A free account covers only a handful of conversations; `402` in the log means out of credit. Benched tokens are cleared by `docker compose restart app`.

10. Testing Performed

10.1 Automated — 203 tests

File	Tests	Covers
`test_tool_calling.py`	58	Loop, scoping, tool offers, argument validation, retries, **token failover**, ordering/cancelling tool guards, order form hand-off
`test_order_drafts.py`	50	Quotes, earlier-turn rule, expiry, price change + savepoint, one-time use, variant forms, form confirm/discard (real Postgres)
`test_catalog.py`	42	Attribute and variant validation, type definitions, slugs
`test_order_cancellations.py`	22	Two-step cancellation, wording, admins limited to own orders (real Postgres)
`test_chat_ws.py`	19	History, WebSocket auth, re-check, reset, bad frames, errors
`test_variants.py`	12	Variant ordering, price override, stock, cancel restock, SKU lookup/uniqueness, in-stock SQL (real Postgres)

10.2 Live (real Docker stack and model)

Scenario	Result
WebSocket conversation with a follow-up ("how many are in stock?")	Resolved from the earlier turn
"What kinds of products do you sell?" / "books by Maya Rivera" / "clothing under $30 in stock"	Correct tools and answers
"Is the USB-C hub available?"	Initially "not listed" (sold out, filtered); fixed — now "currently sold out"
"I want to buy 2 black T-shirts in M... go ahead"	Quoted only (no order in the same turn); first try picked size L, fixed via prompt; retry correct
Order form endpoints	Wrong total refused, changed variant/quantity placed, second confirm refused
"I'd like to order a hoodie"	Form shown immediately after the worked-example prompt change
Variant ordering and cancel via REST	Missing variant 422, sold-out 409, stock 20 → 18 → 20
"Order the T-shirt in white"	Bot picked the white M variant; form listed all four variants
Token failover	Token 1 at 402 → request served by token 2

10.3 Model comparison for tool calling

`phi-4`: no provider supports tools. `Qwen2.5-72B-Instruct`: correct tool (or none) on every test question — adopted. `Llama-3.1-8B-Instruct`: invented tool calls and looped — rejected.

11. Issues Encountered and How They Were Resolved

Issue	Root cause	Resolution
`pip install` blocked	Managed system Python	Project `.venv`
HF `403` / `400 model_not_supported`	Missing token scope; unhosted model	New token; model switch
Ports 5433 / 8000 in use	Other projects' containers	DB port unpublished; API on 8020
"vector type not found"	Table created only by the seed	Knowledge base self-provisions
`phi-4` couldn't call tools	Providers lack tool support	Tested 3 models; switched to Qwen2.5-72B
"Free tier" stated in docs	Never verified	Router listing checked: no free providers; docs corrected
Follow-up questions ignored history	FAQ text in the latest user message dominated "it"	FAQ moved to the system message; previous question added to FAQ search
App wouldn't start after a rebuild (`No module named 'psycopg'`)	Unpinned SQLAlchemy upgraded to 2.1, whose default PG driver is psycopg 3	`sqlalchemy_url()` names `psycopg2`
`FOR UPDATE` with the new joined type relationship	Postgres refuses `FOR UPDATE` on the nullable side of an outer join; would also lock the shared type row	`innerjoin=True`; `with_for_update(of=Product)`

Issue	Root cause	Resolution
Product type schema edits could race with product writes	Validation read the schema unlocked	Product writes share-lock the type row; type updates take an exclusive lock
"Backup drive" search returned the Free plan	OR-matching sorted by price; plan text mentions "backups"	Rank by number of matching terms
Sold-out hub reported as "not listed"	Model filtered to in-stock for "available?"	Tool reports `sold_out_matches` when filtering
"Yes, cancel it" would be rejected	"cancel" was in the negative-word list	Per-action confirmation patterns
Tool guard test failed on a description	Parameter text mentioned "customer"	Reworded (the guard is intentional)
Chat failures looked like random 503s	Provider errors weren't logged; real cause was `402` (credits used up)	Errors logged; token failover added
Second token returned 403	Missing "Inference Providers" permission	Permission added by the user
Bot quoted size L for "size M"	Model picked the wrong variant SKU	Prompt rule to match size/colour/format
Bot asked "which size?" instead of showing the form	An older "ask if unclear" rule; quote seen as risky	Prompt + tool description changed; worked example added
Reply was a bare "[Order Form]"	Model said almost nothing when a form was attached	Standard sentence substituted
Variant silently ignored when ordering a product without variants	`create_order` didn't reject an unknown variant SKU	Now refused; caught by a new test
`load_dotenv()` failed in piped scripts	It inspects the calling frame	Container already has the env; not needed

12. Known Limitations and Open Items

Security 1. No rate limiting on login/registration; registration open to anyone. 2. Plain HTTP in the dev stack. 3. No token revocation, refresh, logout, password change/reset or email verification. 4. Admins only via the seed or the database; no audit log. 5. Customer isolation lives in application code (consider Row-Level Security). 6. Single symmetric JWT key (HS256). 7. Tool results are untrusted text; mitigated by the prompt, and write actions additionally need server-checked confirmation.

Store 8. No payments, shipping addresses, tax, discounts, currency or refunds; buying a plan doesn't change `customers.plan`. 9. Exact stock levels are public. 10. Stock is held by pending orders until cancelled; no reservation timeout. 11. **Variants in JSONB:** SKU uniqueness and references are enforced by the app (advisory locks), not the database; removing a variant that's on orders prevents restocking on cancel (deactivate instead); price filters use the product's base price. 12. Legacy rows: migrated orders point at inactive legacy products; converted per-size products remain as inactive `…-OLD<id>` rows. 13. Migration `0002` is irreversible; migrations run at startup.

Chatbot 14. The "clear yes" check is English-only. 15. WebSocket history lives in memory (lost on restart/reconnect; the page doesn't resend it). 16. The signed-out gate is regex-based and also blocks policy questions containing "my order" (e.g. "Can I cancel my order after it starts processing?"). 17. The model may still skip a tool, pick the wrong variant (the quote/form makes this visible before anything is ordered), or ask a question instead of showing the form for vague requests. 18. `sources` lists retrieved FAQ files even

when the answer came from a tool. 19. Small English embedding model; full FAQ re-index on every startup. 20. Every message costs HF credits; tool calls multiply it. 21. The FAQ's store policies (shipping prices, return window, payment methods) are placeholders to replace with real ones.

Engineering 22. The vector store uses its own (unpooled) connections. 23. Dev-mode container (`--reload`, bind mount, hardcoded DB password). 24. No CI; tests run manually. 25. Unpinned major versions in `requirements.txt` (the SQLAlchemy 2.1 incident).

13. Suggested Next Steps

1. **Before real customers:** rate limiting, TLS, token refresh/revocation, password reset.
2. **WhatsApp channel** (Meta Cloud API): verified phone-number linking, Postgres-stored conversation history, webhook signature checks.
3. **Persist conversation history** for the WebSocket too, so reconnects keep context.
4. **Loosen the signed-out gate** so general policy questions are answered and only real lookups require login.
5. **Server-side fallback** that creates the order form whenever purchase intent is clear and a product was found, regardless of the model's choice.
6. **Payments** and linking purchased plans to `customers.plan`.
7. **Admin tooling and audit log**; CI running the test suite.
8. **Pin dependency majors**; production container image; pooled connections for the vector store.
9. Consider a `product_variants` table if the catalog grows large or other systems write to the database directly.